

Bachelor in Digital Sciences
Final Certification Box - 2025/2026

TECHNICAL DOCUMENTATION

Travel Planning

“Japan Trip Planner”

Team members:

Marc Maalouf · Léo Roussel-Coussin · Lucien Paul Constant

1. Introduction and scope

This document is the **technical documentation** deliverable described in section 4 of the subject. It covers the system architecture, the technology choices and their justification, the interpretations adopted for the subject, the inner workings of the planning algorithm, and the installation and start-up instructions. The **user manual** and the **test protocol** are provided as separate documents.

The application meets the following need: it lets an authenticated user build lists of places, automatically generate a good-quality route that minimises the total distance travelled, keep several results, and share them according to a chosen confidentiality level (public or private).

Git link: <https://github.com/Leorc9/box-certif>

1.1 Language convention

In line with the subject's constraints, **the entire source code and in-code documentation (comments, docstrings) is written in English**. This document is also written in English; identifiers, file names and code snippets are reproduced verbatim.

2. Interpretations adopted for the subject

The subject leaves several points open to interpretation. The following decisions were made and form the functional contract of the application.

Point of the subject	Adopted interpretation
Notion of a “tour”	A tour is a closed cycle visiting each place exactly once and returning to the starting point (Travelling Salesman Problem / TSP).
Score of a tour	Total distance of the cycle, computed with the great-circle formula imposed by the subject (spherical law of cosines, $R = 6378.197$ km).
“Maximum number of places”	All places entered by the user are visited. Optimisation acts on the visiting order, not on selecting a subset.
Retrieving coordinates	Search by place name through the GeoNames API; only latitude and longitude (decimal degrees) are stored.
Geographic grouping	Nearby cities (< 150 km) are grouped around a hotel city and visited as day trips, in order to limit hotel changes — a realistic interpretation of a journey.
“Public” sharing	Any visitor with the link can access the trip, without authentication.
“Private” sharing	The link alone is not enough: the visitor must be logged in and be the owner or a user explicitly targeted (by email).
Account isolation	Each user only sees and edits their own places and trips (ownership check on every operation).

3. System architecture

The application follows a clearly separated **three-tier architecture**: a presentation/application layer in PHP, a stand-alone algorithmic service in Python (FastAPI), and a MySQL persistence layer. This separation allows the algorithm to evolve independently from the interface and lets each block be tested in isolation.

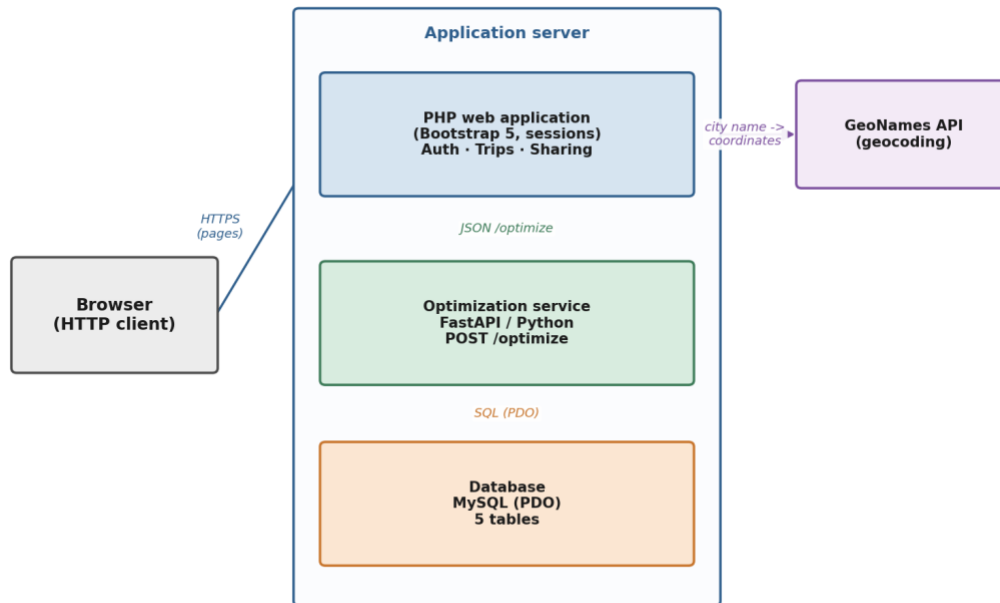


Figure 1 - Three-tier architecture of the application

3.1 Components

Component	Role	Technology
Web application	UI, authentication, trip and sharing management, orchestration	PHP 8 + Bootstrap 5
Optimization service	Computes the optimal tour (clustering + ACO), exposed over HTTP	Python 3 + FastAPI
Database	Persists users, places, trips and shares	MySQL (PDO access)
Geocoding	Converts a city name into coordinates	External GeoNames API

3.2 Repository layout

```

algo/           # Optimization service (Python)
  place.py      # Place value object (name + coordinates)
  distance.py   # Great-circle distance (subject's formula)
  cluster.py    # Grouping of nearby cities (clustering)
  aco.py        # Ant Colony Optimization (TSP)
  api.py        # FastAPI server (/optimize endpoint)
  main.py       # Command-line demonstration
  test.py       # Unit tests for the algorithm
web/           # Web application (PHP)
  config/database.php # PDO connection
  auth/         # signUp.php, logIn.php, logOut.php
  trip/         # index.php (CRUD), editTrip.php
  share.php     # Viewing a shared trip
  shareLink.php # Managing shares (public / private)
  functions/    # loginCheck.php (access guard)
  
```

```
header.php, index.php, needLogin.php, style/, assets/
database/db.sql      # Database schema
tests/              # Slots for additional tests
```

3.3 Flow of an optimisation

1. The user opens a trip and clicks “Optimise the route” (editTrip.php).
2. The PHP server reads the trip's places from the database, builds a JSON payload and calls the Python service over cURL on POST /optimize.
3. The service runs the clustering then the ACO and returns the order of hotel cities, the associated day trips and the distances.
4. The PHP server updates visit_order in trip_places and total_distance_km on the trip, then displays the itinerary.

4. Technology choices and justification

The subject leaves the choice of language and framework free, provided it is justified. Since no technology is favoured, the choices prioritise **readability**, **fitness for purpose** and **separation of responsibilities**.

Choice	Justification	Alternative considered
Python for the algorithm	Mature scientific ecosystem, readability, ideal for clearly expressing the clustering and the ACO.	C++ (faster but oversized here)
FastAPI	Minimal HTTP exposure, automatic input validation via Pydantic, generated OpenAPI docs. Decouples the algorithm from the UI.	Flask (no native validation), CLI only
PHP for the web	Simple server-side rendering, native sessions, immediate deployment on a standard Apache/PHP server.	Node.js/Express, Python/Django
MySQL via PDO	Relational model suited to the users/trips/places relations; PDO provides prepared statements (anti-injection).	SQLite (single file), PostgreSQL
Bootstrap 5	Responsive, readable UI with low CSS cost, consistent with the time available.	Custom CSS (more time-consuming)
GeoNames	Free geocoding API by city name, meets the “retrieve coordinates via an API” requirement.	Nominatim/OpenStreetMap

5. Data model

The database contains five tables. Referential integrity is enforced by foreign keys with cascading deletes (ON DELETE CASCADE).

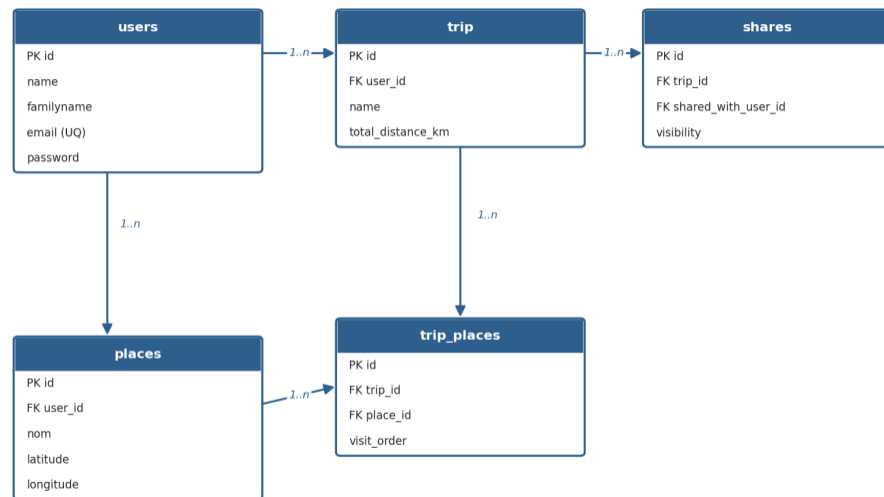


Figure 3 - Relational data model (keys, relations)

Table	Description / notable constraints
users	Accounts: name, familyname, email (UNIQUE), password (hashed). Login identity.
places	A user's places: nom, latitude, longitude. FK user_id (cascade).
trip	A user's trip: name, total_distance_km. FK user_id (cascade).
trip_places	Join table trip $\rightrightarrows$ places, carrying visit_order (computed visiting order).
shares	Shares: visibility ('public' 'private'), shared_with_user_id. CHECK constraint: private $\Rightarrow$ targeted user, public $\Rightarrow$ none. UNIQUE key (trip_id, shared_with_user_id).

6. How the algorithm works

The core of the project is a **two-phase** planner: a geographic clustering first reduces the problem, then an Ant Colony Optimization (ACO) metaheuristic orders the hotel cities. The optimised criterion is the total distance.

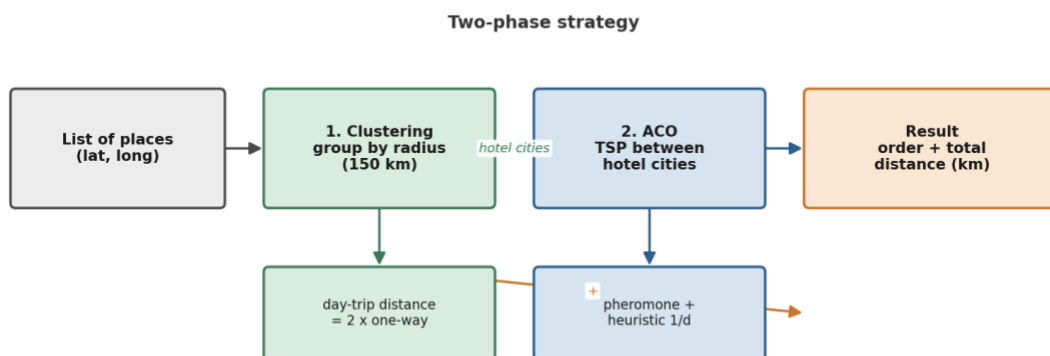


Figure 2 - Planning pipeline: clustering then tour optimization

6.1 Distance between two places

The distance used is the one imposed by the subject: the **great-circle distance** (spherical law of cosines), with coordinates converted to radians.

```
D(A, B) = R · arccos( sin(latA)·sin(latB)
                    + cos(latA)·cos(latB)·cos(lonB - lonA) )
with R = 6378.197 km
```

Robustness: the argument of arccos is clamped to [-1, 1] before the call, because floating-point rounding can push it slightly beyond and make math.acos crash. The distance from a place to itself therefore correctly evaluates to 0.

6.2 Phase 1 — Geographic clustering

Goal: limit hotel changes by grouping nearby cities. The algorithm is **greedy**: it takes the first untreated city as a hotel city, attaches to it every remaining city located within maxRadius (150 km by default), then repeats with the unattached cities.

A cluster's day-trip distance is counted as a **round trip** ($\times 2$) from the hotel city, since the traveller returns to sleep at the same place. Complexity: $O(n^2)$ distance comparisons, negligible compared with the ACO.

6.3 Phase 2 — Ant Colony Optimization (ACO)

The visiting order of the **hotel cities** is a Travelling Salesman Problem (TSP), solved with the **Ant System** algorithm. At each iteration, several ants each build a tour; good tours deposit more pheromone, which gradually steers the colony towards short edges.

Building a tour

From the current city, an ant chooses the next unvisited city by a draw proportional (roulette wheel) to a weight combining two terms:

$$weight(j) = \tau(i,j)^\alpha \cdot (1 / d(i,j))^\beta$$

- **τ (pheromone)**: collective memory; weighted by exponent α (weight of the trail).
- **$1/d$ (heuristic)**: nearer cities are more attractive; weighted by exponent β .

Pheromone update

- **Evaporation**: each edge loses a fraction ρ of its pheromone, which prevents the colony from getting stuck on an early solution.
- **Deposit**: each ant deposits $q / \text{tour_length}$; a shorter tour therefore reinforces its edges more strongly. The matrix is symmetric ($d(i,j) = d(j,i)$).

Chosen hyper-parameters

Parameter	Value	Role
nAnts	20	Number of ants per iteration
nIterations	100	Number of iterations
alpha (α)	1.0	Weight of the pheromone
beta (β)	3.0	Weight of the proximity heuristic
evaporation (ρ)	0.5	Evaporation rate

Parameter	Value	Role
q	100.0	Pheromone quantity deposited per tour

Algorithmic complexity

The distance matrix is precomputed once in $O(n^2)$. The core of the ACO has complexity **$O(n_{\text{iterations}} \times n_{\text{Ants}} \times n^2)$** : each ant builds a tour in $O(n^2)$ (at each step, scanning the candidate cities), repeated for all ants and all iterations. The prior clustering reduces n (number of hotel cities < number of places), which lowers the cost of the ACO accordingly.

Why it beats a naive solution: compared with a random order or a simple “nearest neighbour”, the ACO explores the solution space in a guided way and keeps the best tour found, producing a noticeably shorter route while remaining computable within the allotted time.

6.4 Optimisation API contract

The service exposes a single endpoint.

	Description
Method / route	POST /optimize
Input	JSON: places (list of { name, latitude, longitude }) and maxRadius (km, default 150).
Validation	≥ 2 cities required, otherwise HTTP 400; types validated by Pydantic.
Output	route (hotel order), clusters (hotel + day trips), interHotelDistance, dayTripDistance, totalDistance.

```
// Example request
POST /optimize
{
  "places": [
    {"name": "Tokyo", "latitude": 35.6820, "longitude": 139.7622},
    {"name": "Kyoto", "latitude": 34.9946, "longitude": 135.7344}
  ],
  "maxRadius": 150
}
```

7. Security

Several measures protect the application against common web vulnerabilities.

Risk	Mitigation in place
SQL injection	All queries use PDO prepared statements with bound parameters; no concatenation of user values.
Passwords	Hashing via password_hash() (bcrypt); verification with password_verify(). No plaintext passwords.
XSS	Systematic escaping of dynamic output with htmlspecialchars().
Access control	PHP sessions; checkLogin() guard on protected pages; ownership check (user_id) on every trip/share operation.
Isolation	A user can only read/edit their own data; private sharing requires login + explicit targeting.

8. Testing

The algorithm — the critical part of the project — is covered by **unit tests** in `algo/test.py`. Seven cases cover the `Place` object, the distance (zero, symmetry, Tokyo–Kyoto order of magnitude ≈ 370 km), the distance matrix, a tour's length, building a valid tour, selecting the next city, and a full run of the ACO (with a fixed seed for reproducibility).

```
$ cd algo && python3 test.py
Place ..... OK   Distance ..... OK   Matrix ..... OK
Tour length ... OK   Build tour .... OK   Choose next ... OK
Run ..... OK
All tests passed!
```

The web features (authentication, sharing, ownership checks) are covered by a **manual test protocol** described in the dedicated deliverable.

9. Installation and start-up

9.1 Prerequisites

- PHP 8 with the cURL extension, and a web server (Apache, or the built-in `php -S` server).
- MySQL / MariaDB.
- Python 3.10+ with `pip`.
- A GeoNames account (username to fill in) for geocoding.

9.2 Database

```
mysql -u root -p -e "CREATE DATABASE boxdb;"
mysql -u root -p boxdb < database/db.sql
```

Then fill in the connection credentials in `web/config/database.php` (variables `$host`, `$dbname`, `$username`, `$password`).

9.3 Optimization service (Python)

```
cd algo
pip install fastapi uvicorn pydantic
uvicorn api:app --host 127.0.0.1 --port 8000
```

The service then listens on `http://127.0.0.1:8000`; the interactive documentation is available at `/docs`. The algorithm can also be run on its own as a demo via `python3 main.py`.

9.4 Web application (PHP)

```
cd web
php -S localhost:8080
```

Open `http://localhost:8080/index.php`. Create an account (Sign Up), log in, create a trip, add cities by name, then click “Optimise the route” (the Python service must be running).

9.5 Geocoding configuration

In `web/trip/editTrip.php`, replace the `username` parameter of the GeoNames URL with a valid account identifier. Without it, the city search will fail.

10. Known limitations and future work

In the spirit of maintenance documentation, the identified limitations and possible improvements are listed below.

- **Case of include file names:** some pages include `config/dataBase.php` while others include `config/database.php`. On a case-insensitive file system (macOS, Windows) this works, but it may fail on Linux. Unifying the name (`database.php`) makes deployment more reliable.
- **Access guard:** `loginCheck.php` redirects without `exit()` after `header()`; adding `exit()` prevents the rest of the page from running after the redirect.
- **Optimize endpoint reachable from public shares:** the `optimize` action in `share.php` can be triggered by any visitor of a public link; restricting it to the owner would strengthen control.
- **Automated web tests:** complementing the manual protocol with integration tests (e.g. via PHPUnit) would increase coverage.
- **Persisting optimisation results:** the optimised order is stored, but the hotel/day-trip breakdown is only recomputed at display time; persisting it would avoid recomputation.
- **Externalised configuration:** moving DB credentials, the Python service URL and the GeoNames key into environment variables would improve portability and security.